

脑电运动解码入门环境配置与使用说明

2026 年 9 月 15 日

1 环境用途

本环境用于入门级脑电信息解码实验，尤其适合先做运动想象（Motor Imagery, MI）任务，例如左手与右手运动想象二分类。环境已将主要软件包、数据集目录、缓存目录配置到 F 盘，避免公开数据集和 Python 包大量占用 C 盘。

推荐的入门实验流程如下：

1. 使用公开运动想象 EEG 数据集。
2. 用 MNE 做读取、滤波、分段和可视化。
3. 用 CSP、LDA、SVM 或黎曼几何方法做分类。
4. 用交叉验证评估准确率。
5. 后续再升级到深度学习方法，例如 EEGNet、Braindecode。

2 目录结构

当前配置的主要目录如下：

3 已安装的软件包

环境名称为 `eeg-mi`，Python 版本为 3.11.15。已验证的软件包包括：

4 环境变量

以下路径已经写入系统环境变量和 Conda 环境变量。重新打开终端后生效。

路径	作用
F:\Miniforge3	Conda/Miniforge 管理器安装位置
F:\eeg_bci\envs\eeg-mi	脑电实验 Python 环境
F:\eeg_bci\data	EEG 数据集根目录
F:\eeg_bci\data\mne	MNE 下载数据目录
F:\eeg_bci\data\moabb	MOABB 下载数据目录
F:\eeg_bci\projects	建议存放代码、Notebook 和实验结果
F:\eeg_bci\pkgs	Conda/Pip 包缓存目录
F:\eeg_bci\tmp	临时文件目录

软件包	版本	用途
mne	1.12.1	EEG/MEG 数据读取、预处理、滤波、分段、可视化
moabb	1.5.0	公开 BCI 数据集下载与统一评测
scikit-learn	1.9.0	机器学习分类器、交叉验证、指标评估
pyriemann	0.12	基于黎曼几何的 EEG 分类方法
jupyterlab	已安装	交互式编程和实验记录

Listing 1: 已配置的环境变量

```
MNE_DATA=F:\eeg_bci\data\mne
MOABB_DATA=F:\eeg_bci\data\moabb
JOBLIB_TEMP_FOLDER=F:\eeg_bci\tmp
MPLCONFIGDIR=F:\eeg_bci\tmp\matplotlib
PIP_CACHE_DIR=F:\eeg_bci\pkgs\pip-cache
```

这些变量的作用是让 MNE、MOABB、Matplotlib、Joblib 和 Pip 优先使用 F 盘目录。

5 启动 JupyterLab

平时建议使用 JupyterLab 做脑电实验。打开 PowerShell，运行：

Listing 2: 启动 JupyterLab

```
F:\Miniforge3\Scripts\conda.exe run -p F:\eeg_bci\envs\eeg-mi jupyter lab --
notebook-dir F:\eeg_bci\projects
```

运行后终端会显示一个类似下面的地址：

```
http://localhost:8888/lab?token=...
```

在浏览器中打开该地址即可使用。Notebook 和实验代码建议保存在：

```
F:\eeg_bci\projects
```

6 运行 Python 脚本

如果不使用 JupyterLab，也可以直接运行某个 Python 文件：

Listing 3: 运行 Python 文件

```
F:\Miniforge3\Scripts\conda.exe run -p F:\eeg_bci\envs\eeg-mi python F:\eeg_bci\projects\test.py
```

7 环境测试代码

在 JupyterLab 中新建 Notebook，运行下面代码。如果能够输出版本号，说明环境可用。

Listing 4: 验证核心软件包

```
import os
import mne
import moabb
import sklearn
import pyriemann

print("mne:", mne.__version__)
print("moabb:", moabb.__version__)
print("sklearn:", sklearn.__version__)
print("pyriemann:", pyriemann.__version__)
print("MNE_DATA:", os.environ.get("MNE_DATA"))
print("MOABB_DATA:", os.environ.get("MOABB_DATA"))
```

8 第一个运动想象数据集测试

可以先测试 MOABB 是否能识别公开数据集。例如使用经典的 BNCI2014_001 数据集：

Listing 5: 查看公开运动想象数据集信息

```
from moabb.datasets import BNCI2014_001
```

```
dataset = BNCI2014_001()

print("Subjects:", dataset.subject_list[:5])
print("Events:", dataset.event_id)
```

该数据集包含左手、右手、脚、舌头等运动想象任务。入门时建议先只做左手与右手二分类。

9 推荐入门实验路线

建议先按下面顺序推进：

1. 跑通 BNCI2014_001 数据集读取。
2. 只选择左手和右手两类。
3. 对 EEG 进行 8--30 Hz 滤波。
4. 取提示后约 0.5--4 s 的时间窗。
5. 使用 CSP 提取空间特征。
6. 使用 LDA 或 SVM 分类。
7. 使用交叉验证报告准确率和平衡准确率。

不要一开始就直接做深度学习。先用 CSP + LDA 建立一个稳定基线，后续再比较 EEGNet、ShallowConvNet 等方法。

10 后续升级

如果以后要做深度学习，可以在当前环境中追加安装 PyTorch 和 Braindecode：

Listing 6: 升级到深度学习环境

```
F:\Miniforge3\Scripts\conda.exe run -p F:\eeg_bci\envs\eeg-mi pip install -i
https://pypi.tuna.tsinghua.edu.cn/simple torch braindecode
```

如果后续需要 GPU 版 PyTorch，应根据电脑显卡和 CUDA 版本选择对应安装命令。没有 GPU 时也可以先使用 CPU 版，入门实验和传统机器学习方法不依赖 GPU。

路径	大约占用
F:\Miniforge3	0.62 GB
F:\eeg_bci\envs\eeg-mi	1.97 GB
F:\eeg_bci\pkgs	2.33 GB
F:\eeg_bci\data	当前约 0 GB

11 内存与磁盘建议

当前入门环境大约占用如下空间：

建议配置：

- 最低内存：8 GB。
- 推荐内存：16 GB 或以上。
- 初期硬盘空间：至少预留 10--20 GB。
- 多数据集实验：建议预留 50 GB 或以上。

降低内存占用的做法：

Listing 7: 降低 EEG 处理内存占用的常用做法

```
raw = mne.io.read_raw_fif(path, preload=False)
raw.pick_types(eeg=True)
raw.filter(8, 30)
raw.resample(128)
```

做 MOABB 实验时，也建议先单个被试、单个数据集跑通，不要一开始批量跑所有被试和所有数据集。

12 常见问题

12.1 是否容易升级？

容易。当前环境已经包含 MNE、MOABB、Scikit-learn 和 Pyriemann，适合传统运动想象解码。后续只需要在同一环境中追加 PyTorch、Braindecode 或其他库即可。

12.2 会不会占 C 盘？

主要环境、包缓存和数据集目录都已经放在 F 盘。C 盘可能仍会保存少量 Windows 用户配置文件，但不会是主要数据和包存放位置。

12.3 为什么不用系统自带 Python?

系统 Python 当前是 3.12.3。脑电分析工具链里有些包对版本和依赖比较敏感，使用独立 Conda 环境可以减少冲突，也方便以后升级、删除或复现实验。

12.4 以后怎么删除这个环境?

如果以后确认不需要该环境，可以执行：

Listing 8: 删除 eeg-mi 环境

```
F:\Miniforge3\Scripts\conda.exe env remove -p F:\eeg_bci\envs\eeg-mi
```

删除前应先确认自己的代码和数据是否已经备份。一般情况下不建议删除下面目录中的项目文件：

```
F:\eeg_bci\projects
```

13 入门实验操作步骤与原理

本节给出第一个完整实验：使用公开数据集 BNCI2014_001，完成左手与右手运动想象二分类。目标是先跑出一个可靠的传统机器学习基线：

```
EEG数据 -> 8-30 Hz滤波 -> 0.5-4.0 s分段 -> CSP特征 -> LDA分类 -> 交叉验证准确率
```

13.1 启动 JupyterLab

打开 PowerShell，运行：

Listing 9: 启动入门实验 Notebook

```
F:\Miniforge3\Scripts\conda.exe run -p F:\eeg_bci\envs\eeg-mi jupyter lab --  
notebook-dir F:\eeg_bci\projects
```

在浏览器中打开终端给出的 JupyterLab 地址，新建 Notebook，建议命名为：

```
motor_imagery_baseline.ipynb
```

13.2 第一步：测试环境

在第一个 Notebook 单元中运行：

Listing 10: 测试脑电实验环境

```
import os
```

```
import mne
import moabb
import sklearn
import pyriemann

print("mne:", mne.__version__)
print("moabb:", moabb.__version__)
print("sklearn:", sklearn.__version__)
print("pyriemann:", pyriemann.__version__)
print("MNE_DATA:", os.environ.get("MNE_DATA"))
print("MOABB_DATA:", os.environ.get("MOABB_DATA"))
```

如果能够输出版本号，并且 MNE_DATA、MOABB_DATA 指向 F 盘，说明环境正常。

13.3 第二步：查看公开数据集

Listing 11: 查看 BNCI2014_001 数据集信息

```
from moabb.datasets import BNCI2014_001

dataset = BNCI2014_001()

print("被试列表:", dataset.subject_list)
print("任务标签:", dataset.event_id)
```

应能看到类似结果：

```
{'left_hand': 1, 'right_hand': 2, 'feet': 3, 'tongue': 4}
```

入门实验先只使用 left_hand 和 right_hand 两类。

13.4 第三步：读取一个被试的数据

第一次运行下面代码时会下载数据。由于前面已经设置了环境变量，数据会保存到 F 盘的数据目录中。

Listing 12: 读取一个被试的左右手运动想象数据

```
from moabb.paradigms import MotorImagery

paradigm = MotorImagery(
    events=["left_hand", "right_hand"],
    fmin=8,
    fmax=30,
```

```
tmin=0.5,
tmax=4.0,
resample=128
)

X, y, metadata = paradigm.get_data(
    dataset=dataset,
    subjects=[1]
)

print("X_shape:", X.shape)
print("y_shape:", y.shape)
print("标签:", set(y))
print(metadata.head())
```

X 的形状通常是:

试次数 x 通道数 x 时间点数

例如 (144, 22, 448) 表示: 144 个运动想象片段、22 个 EEG 通道、每段 448 个采样点。

13.5 第四步: CSP + LDA 分类

Listing 13: CSP + LDA 交叉验证分类

```
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from mne.decoding import CSP

clf = make_pipeline(
    CSP(n_components=6, reg=None, log=True, norm_trace=False),
    LinearDiscriminantAnalysis()
)

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

```
scores = cross_val_score(  
    clf,  
    X,  
    y,  
    cv=cv,  
    scoring="balanced_accuracy"  
)  
  
print("每折准确率:", scores)  
print("平均准确率:", scores.mean())
```

如果平均准确率明显高于 0.5，说明模型已经学到了一定的左右手运动想象差异。

13.6 第五步：绘制混淆矩阵

Listing 14: 绘制混淆矩阵

```
from sklearn.model_selection import cross_val_predict  
from sklearn.metrics import ConfusionMatrixDisplay  
import matplotlib.pyplot as plt  
  
y_pred = cross_val_predict(clf, X, y, cv=cv)  
  
ConfusionMatrixDisplay.from_predictions(y, y_pred)  
plt.title("Left_hand_vs_Right_hand")  
plt.show()
```

混淆矩阵可以显示左手和右手分别被正确分类、错误分类的次数。它比单独看一个平均准确率更直观。

13.7 实验原理

运动想象解码不是直接判断原始 EEG 波形是否相似，而是利用运动想象在特定脑区和特定频段引起的节律能量变化。

左右手运动想象主要涉及感觉运动皮层：

- 左手运动想象通常更明显影响右侧运动皮层。
- 右手运动想象通常更明显影响左侧运动皮层。
- 常关注的节律包括 8--13 Hz 的 Mu 节律和 13--30 Hz 的 Beta 节律。

因此入门实验中使用 8--30 Hz 滤波，目的是保留和运动想象最相关的频段，同时减少低频漂移和高频噪声。

时间窗设置为 0.5--4.0 s，表示从提示出现后 0.5 秒开始截取到 4 秒。这样可以避开刚看到提示时的瞬时视觉反应和准备阶段，使分类特征更集中在运动想象过程。

13.8 CSP 的含义

CSP 是 Common Spatial Pattern，中文可理解为公共空间模式。它的目标是学习一组空间滤波器，使两类任务的方差差异尽可能大。

直观理解：

找到一些 EEG 通道的加权组合，
让左手想象时能量较强、右手想象时能量较弱，
或者让右手想象时能量较强、左手想象时能量较弱。

EEG 单个通道信噪比较低，所以 CSP 不只依赖某一个电极，而是自动学习多个电极的组合。CSP 输出的是每个 trial 的空间模式能量特征，而不是原始波形。

13.9 LDA 的含义

LDA 是 Linear Discriminant Analysis，即线性判别分析。它会在 CSP 提取出的特征空间中寻找一条线性分类边界，把左手和右手尽量分开。

LDA 适合作为入门基线，原因是：

- 对小样本 EEG 数据比较友好。
- 计算量小，结果稳定。
- 与 CSP 搭配是运动想象 BCI 中非常经典的传统方法。

13.10 为什么要用交叉验证

不能只在训练数据上报告准确率，因为这样会高估模型性能。交叉验证会把数据分成若干份，例如 5 份：

每次用 4 份训练，1 份测试；
重复 5 次；
最后报告平均测试准确率。

代码中使用 `make_pipeline(CSP, LDA)` 的原因是：每一折交叉验证中，CSP 只会在训练集上学习空间滤波器，然后再应用到测试集。这样可以避免数据泄漏。

13.11 完成本实验后的扩展方向

跑通第一个 baseline 后，可以按下面顺序扩展：

1. 更换不同被试，观察每个被试的准确率差异。
2. 调整频段，例如 8--13 Hz、13--30 Hz、8--35 Hz。
3. 调整时间窗，例如 1.0--4.0 s 或 0.5--3.0 s。
4. 尝试 FBCSP，即多个频段分别做 CSP 后再分类。
5. 使用 PyRiemann 方法，与 CSP + LDA 对比。
6. 最后再升级到 Braindecode、EEGNet 等深度学习方法。

14 遇到问题和解决方法

本节记录配置和使用过程中已经遇到的问题。遇到类似现象时，优先按本节排查。

14.1 Select Kernel 中没有 eeg-mi

如果 JupyterLab 的 Select Kernel 列表中没有看到 eeg-mi 或 Python (eeg-mi)，不一定代表环境不能用。先在 Notebook 中运行：

Listing 15: 确认当前 Notebook 使用的 Python 解释器

```
import sys
print(sys.executable)
```

如果输出类似：

```
F:\eeg_bci\envs\eeg-mi\python.exe
```

说明当前 Notebook 实际上已经在使用正确的脑电环境。即使 Kernel 名字显示为 Python 3，也可以继续实验。

进一步验证核心包：

Listing 16: 确认脑电包可用

```
import mne
import moabb

print(mne.__version__)
print(moabb.__version__)
```

如果可以正常输出版本号，说明环境可用。

14.2 手动注册 Python (eeg-mi) Kernel

如果希望 Select Kernel 中明确显示 Python (eeg-mi), 可以关闭当前 JupyterLab 后, 在 PowerShell 中运行下面命令。

先创建 Jupyter 相关目录:

Listing 17: 创建 Jupyter 配置目录

```
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\config"
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\data"
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\runtime"
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\tmp"
```

然后设置当前 PowerShell 会话的 Jupyter 路径:

Listing 18: 设置 Jupyter 和临时目录到 F 盘

```
$env:JUPYTER_CONFIG_DIR="F:\eeg_bci\jupyter\config"
$env:JUPYTER_DATA_DIR="F:\eeg_bci\jupyter\data"
$env:JUPYTER_RUNTIME_DIR="F:\eeg_bci\jupyter\runtime"
$env:TMP="F:\eeg_bci\tmp"
$env:TEMP="F:\eeg_bci\tmp"
```

注册内核:

Listing 19: 注册 eeg-mi 内核

```
F:\eeg_bci\envs\eeg-mi\python.exe -m ipykernel install --sys-prefix --name eeg-
mi --display-name "Python (eeg-mi)"
```

成功后通常会出现类似输出:

```
Installed kernelspec eeg-mi in ...
```

之后重新启动 JupyterLab, 再选择 Python (eeg-mi)。

14.3 New-Item: 路径中具有非法字符

如果 PowerShell 报:

```
New-Item : 路径中具有非法字符。
```

通常是复制命令时路径中混入了换行或隐藏字符。例如路径被复制成了类似:

```
F:\
\eeg_bci\jupyter
```

这种路径是非法的。解决方法是: 不要复制很长的一整行命令, 改成一行为一个目录, 手动确认路径中间没有换行:

Listing 20: 逐行创建目录

```
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\config"
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\data"
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\runtime"
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\tmp"
```

也可以使用更短的写法:

Listing 21: 使用 mkdir 创建目录

```
cd F:\
mkdir eeg_bci
mkdir eeg_bci\jupyter
mkdir eeg_bci\jupyter\config
mkdir eeg_bci\jupyter\data
mkdir eeg_bci\jupyter\runtime
mkdir eeg_bci\tmp
```

如果提示目录已经存在, 可以忽略。

14.4 New-Item: 找不到 LiteralPath 参数

有些 PowerShell 环境中 New-Item 不支持 -LiteralPath 参数。如果出现:

```
New-Item : 找不到与参数名称“LiteralPath”匹配的参数。
```

把 -LiteralPath 改成 -Path 即可:

Listing 22: 使用 Path 参数

```
New-Item -ItemType Directory -Force -Path "F:\eeg_bci\jupyter\config"
```

14.5 IPProgress not found 警告

在 Notebook 中导入 MOABB 或运行数据下载时, 可能出现:

```
TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets.
```

这不是错误。它表示 Jupyter 中的进度条组件 ipywidgets 没有安装或没有启用, 影响的是下载或运行时的进度条显示, 不影响 MNE、MOABB、CSP、LDA 等核心实验代码运行。

如果环境测试输出如下内容, 说明环境本身正常:

```
mne: 1.12.1
moabb: 1.5.0
```

```
sklearn: 1.9.0
pyriemann: 0.12
MNE_DATA: F:\eeg_bci\data\mne
MOABB_DATA: F:\eeg_bci\data\moabb
```

若希望消除该警告，可以关闭 JupyterLab 后，在 PowerShell 中运行：

Listing 23: 安装 ipywidgets

```
F:\Miniforge3\Scripts\conda.exe install -p F:\eeg_bci\envs\eeg-mi -c conda-
forge ipywidgets -y
```

安装完成后重新启动 JupyterLab。

14.6 paradigm.get_data() 报错的修复方案

如果运行下面代码时在 `paradigm.get_data()` 附近报错：

```
X, y, metadata = paradigm.get_data(
    dataset=dataset,
    subjects=[1]
)
```

常见原因不是分类算法错误，而是第一次下载或读取 BNCI 数据集时，MNE/MOABB 尝试写入 C 盘用户配置目录，例如：

```
C:\Users\LENOVO\.mne
```

如果该目录权限异常，就会出现 `PermissionError`。此外，MOABB 1.5 在 Windows 下还可能把盘符 F: 中的冒号错误清洗成 F-，导致数据路径跑到当前工作目录下面。

推荐在 Notebook 最前面使用下面的完整修复单元。运行后，建议重启 Kernel，并从第一个单元开始依次运行。

Listing 24: 修复 MNE/MOABB 数据路径与 Windows 盘符问题

```
import os
from pathlib import Path

base = Path(r"F:\eeg_bci")

for p in [
    base / ".mne",
    base / "data" / "mne",
    base / "data" / "moabb",
    base / "tmp",
```

```

    base / "tmp" / "matplotlib",
]:
    p.mkdir(parents=True, exist_ok=True)

os.environ["_MNE_FAKE_HOME_DIR"] = str(base)
os.environ["MNE_DATA"] = str(base / "data" / "mne")
os.environ["MNE_DATASETS_BNCI_PATH"] = str(base / "data" / "mne")
os.environ["MOABB_DATA"] = str(base / "data" / "moabb")
os.environ["MPLCONFIGDIR"] = str(base / "tmp" / "matplotlib")
os.environ["JOBLIB_TEMP_FOLDER"] = str(base / "tmp")

```

然后在导入数据集前加入 MOABB Windows 路径修复:

Listing 25: 修复 MOABB 1.5 的 Windows 路径清洗问题

```

from pathlib import Path
import moabb.datasets.download as moabb_download

moabb_download._sanitize_path = lambda path: Path(path)

from moabb.datasets import BNCI2014_001
from moabb.paradigms import MotorImagery

dataset = BNCI2014_001()

print(dataset.event_id)
print(dataset.subject_list)

```

再读取第 1 个被试的左右手运动想象数据:

Listing 26: 读取 subject 1 的左右手运动想象数据

```

paradigm = MotorImagery(
    n_classes=2,
    events=["left_hand", "right_hand"],
    fmin=8,
    fmax=30,
    tmin=0.5,
    tmax=4.0,
    resample=128
)

X, y, metadata = paradigm.get_data(

```

```

dataset=dataset,
subjects=[1]
)

print("X_shape:", X.shape)
print("y_shape:", y.shape)
print("标签:", set(y))
print(metadata.head())

```

如果输出类似:

```

X shape: (288, 22, 449)
y shape: (288,)
标签: {'right_hand', 'left_hand'}
  subject session run
0  1  0train  0
1  1  0train  0

```

说明数据读取和预处理已经成功。这里 288 表示 trial 数量, 22 表示 EEG 通道数量, 449 表示每个 trial 的时间采样点数。设置 `tmin=0.5`、`tmax=4.0`、`resample=128` 后, 理论时间长度为 3.5 s, MNE 通常会同时包含起止采样点, 因此得到 449 个时间点是正常的。

需要注意: 这一步只是下载、读取和预处理数据, 还没有训练分类模型。真正训练发生在后续的 `cross_val_score` 或 `clf.fit(...)` 中。

14.7 下载进度很慢和 SHA256 提示

第一次读取公开数据集时, 可能出现类似输出:

```

SHA256 hash of downloaded file: ...
Use this value as the 'known_hash' argument ...
11% | 4.64M/42.8M ...

```

SHA256 是文件校验值, 用于确认下载文件是否完整, 不是错误。下载速度慢通常是公开数据服务器或当前网络连接较慢导致的。只要最后能输出 `X shape`、`y shape` 和标签集合, 就说明数据已经读取成功。下次运行同一数据通常会优先使用 F 盘缓存, 不需要重新下载。

14.8 确认数据没有主要写入 C 盘

修复后, 主要数据和缓存应该位于:

```
F:\eeg_bci\data\mne
F:\eeg_bci\data\moabb
F:\eeg_bci\.mne
F:\eeg_bci\tmp
```

可以在 Notebook 中检查:

Listing 27: 检查数据和配置路径

```
import os

print("MNE_DATA:", os.environ.get("MNE_DATA"))
print("MOABB_DATA:", os.environ.get("MOABB_DATA"))
print("MNE_fake_home:", os.environ.get("_MNE_FAKE_HOME_DIR"))
print("JOBLIB_TEMP_FOLDER:", os.environ.get("JOBLIB_TEMP_FOLDER"))
print("MPLCONFIGDIR:", os.environ.get("MPLCONFIGDIR"))
```

这些路径都指向 F 盘时, 公开 EEG 数据集不会大量占用 C 盘。C 盘仍可能有少量系统级配置、浏览器缓存或 Windows 临时文件; 如果运行内存不足, Windows 还可能使用 C 盘页面文件。这不是 EEG 数据主动下载到 C 盘。

14.9 没有 Anaconda, 为什么还能使用 conda

本机没有安装 Anaconda。当前使用的 conda 来自安装在 F 盘的 Miniforge:

```
F:\Miniforge3\Scripts\conda.exe
```

三者区别可以简单理解为:

- Anaconda: 体积较大的 Python/Conda 发行版, 默认包含大量科学计算包。
- Miniconda: 精简版 Conda。
- Miniforge: 基于 conda-forge 社区源的精简 Conda, 适合科研和开源工具链。

因此, 即使系统中直接输入 conda 找不到命令, 也可以通过完整路径使用:

Listing 28: 使用 Miniforge 中的 conda

```
F:\Miniforge3\Scripts\conda.exe --version
```

本实验环境的位置是:

```
F:\eeg_bci\envs\eeg-mi
```

平时启动 JupyterLab 时也使用这个 Miniforge 的 conda.exe:

Listing 29: 启动 JupyterLab

```
F:\Miniforge3\Scripts\conda.exe run -p F:\eeg_bci\envs\eeg-mi jupyter lab --
notebook-dir F:\eeg_bci\projects
```

15 下肢脑电解码入门路线

如果目标是做下肢相关脑电解码，需要先确认数据集中是否有下肢标签。当前入门使用的 BNCI2014_001 是真实公开 EEG 数据集，不是仿真数据。它的事件标签为：

```
{'left_hand': 1, 'right_hand': 2, 'feet': 3, 'tongue': 4}
```

其中 `feet` 表示脚部或双脚运动想象类别，并不是 `left_leg`、`right_leg`、`left_foot`、`right_foot`。因此该数据集适合先做：

- 左手 vs 右手。
- 左手 vs feet。
- 右手 vs feet。
- 手部 vs 脚部。
- 左手、右手、feet、tongue 四分类。

但它不能直接做：

左腿 vs 右腿

因为原始标签中没有左右腿或左右脚的区分。

15.1 BNCI2014_001 数据特点

BNCI2014_001 是经典运动想象 EEG 数据集，适合作为 BCI 入门和算法基线实验。它的主要特点如下：

- 数据来自真实 EEG 采集，不是仿真数据。
- MOABB 中包含 9 名被试。
- 任务标签包括 `left_hand`、`right_hand`、`feet`、`tongue`。
- 常用 EEG 通道数为 22 个。
- 适合做运动想象分类、CSP + LDA 基线、频段分析和 ERD/ERS 分析。

在当前 Notebook 中使用:

```
X, y, metadata = paradigm.get_data(
    dataset=dataset,
    subjects=[1]
)
```

得到的 `x` 是已经按任务事件切分好的 EEG 片段。若输出为:

```
X shape: (288, 22, 449)
y shape: (288,)
```

表示共有 288 个 trial, 每个 trial 有 22 个 EEG 通道, 每个通道有 449 个时间采样点。这里的 449 来自截取 0.5--4.0 s 并重采样到 128 Hz 后的时间点数; MNE 通常会同时包含起止采样点, 因此比 $3.5 \times 128 = 448$ 多 1 个点。

15.2 feet 类别的含义

`feet` 是脚部或双脚运动想象类别。它表示该 trial 中被试按照实验提示进行脚部相关运动想象, 但数据标签没有进一步区分左脚、右脚、左腿或右腿。

因此, 使用 BNCI2014_001 时可以研究:

```
feet vs left_hand
feet vs right_hand
feet vs tongue
left_hand vs right_hand vs feet vs tongue
```

但不能研究:

```
left_foot vs right_foot
left_leg vs right_leg
```

原因是监督学习需要真实标签。若原始数据只有 `feet` 一个下肢标签, 就不能通过代码把它拆成 `left_foot` 和 `right_foot`。强行拆分会造成伪标签, 分类结果没有实验意义。

15.3 用 feet 做什么分类

当前阶段建议先做二分类, 例如:

Listing 30: 提取 `left_hand` vs `feet` 数据

```
paradigm = MotorImagery(
    n_classes=2,
    events=["left_hand", "feet"],
```

```
fmin=8,  
fmax=30,  
tmin=0.5,  
tmax=4.0,  
resample=128  
)
```

这个任务的含义是判断一个 EEG trial 更像左手运动想象，还是更像脚部运动想象。它不是最终的脊髓损伤左右腿解码任务，但可以作为下肢相关解码的入门基线。

也可以做：

Listing 31: 提取 right_hand vs feet 数据

```
paradigm = MotorImagery(  
    n_classes=2,  
    events=["right_hand", "feet"],  
    fmin=8,  
    fmax=30,  
    tmin=0.5,  
    tmax=4.0,  
    resample=128  
)
```

如果后续目标是 SCI 下肢康复，feet 可以作为早期过渡任务，用于学习脑电处理流程、建立 CSP + LDA 基线、观察脚部运动想象在 Cz 附近的 Mu/Beta 节律变化。真正的左右腿或步态想象研究，需要进一步换用带有 left_leg/right_leg 或 gait_imagery/rest 标签的数据集。

15.4 为什么左右腿更难

从脑电原理上看，左右手运动想象相对容易区分，因为左右手对应的感觉运动皮层分别位于对侧半球，空间差异较明显。下肢运动区更靠近大脑中线，左右腿在头皮 EEG 上的空间差异更小，而 EEG 本身空间分辨率有限，所以左右腿或左右脚解码通常比左右手更难。

15.5 建议的下肢研究过渡方案

如果当前阶段主要想做下肢脑电解码，建议先从 feet 类别开始，而不是直接追求左右腿分类。

可以先做二分类：

Listing 32: 左手 vs feet 下肢相关二分类

```

paradigm = MotorImagery(
    n_classes=2,
    events=["left_hand", "feet"],
    fmin=8,
    fmax=30,
    tmin=0.5,
    tmax=4.0,
    resample=128
)

X, y, metadata = paradigm.get_data(
    dataset=dataset,
    subjects=[1]
)

print("X_shape:", X.shape)
print("y_shape:", y.shape)
print("标签:", set(y))

```

也可以做右手 vs feet:

Listing 33: 右手 vs feet 下肢相关二分类

```

paradigm = MotorImagery(
    n_classes=2,
    events=["right_hand", "feet"],
    fmin=8,
    fmax=30,
    tmin=0.5,
    tmax=4.0,
    resample=128
)

```

在该阶段,分类器仍然可以使用 CSP + LDA。区别只是标签从 left_hand/right_hand 换成了 left_hand/feet 或 right_hand/feet。

16 CSP + LDA 与 EEG 数据的数学耦合

本节说明 CSP + LDA 到底在训练什么, 以及它如何与当前提取到的 EEG 数据 x 和标签 y 连接。

16.1 EEG 数据的数学形式

通过 MOABB 得到的数据：

```
X, y, metadata = paradigm.get_data(...)
```

可以看成是一个三维数组：

$$X \in \mathbb{R}^{N \times C \times T}$$

其中：

- N 是 trial 数量，例如 288。
- C 是 EEG 通道数，例如 22。
- T 是每个 trial 的时间采样点数，例如 449。

第 i 个 trial 可以写成矩阵：

$$X_i \in \mathbb{R}^{C \times T}$$

对应标签为：

$$y_i \in \{0, 1\}$$

例如在 `left_hand vs feet` 任务中，可以把 `left_hand` 视为类别 0，把 `feet` 视为类别 1。监督学习的目标是学习一个函数：

$$f(X_i) \rightarrow y_i$$

即输入一段 EEG trial，输出该 trial 属于哪一类运动想象。

16.2 为什么先滤波

运动想象 EEG 主要关注感觉运动节律，包括：

$$\mu : 8-13 \text{ Hz}$$

$$\beta : 13-30 \text{ Hz}$$

因此在代码中设置：

```
fmin=8
fmax=30
```

相当于只保留与运动想象更相关的频段，减少低频漂移和高频噪声。滤波后，CSP 主要在 Mu/Beta 节律上寻找空间差异。

16.3 CSP 的核心思想

CSP 的目标是学习空间滤波器：

$$W \in \mathbb{R}^{K \times C}$$

其中 C 是原始 EEG 通道数， K 是选取的 CSP 分量数，例如代码中的：

```
CSP(n_components=6)
```

对每个 trial，CSP 会进行空间投影：

$$Z_i = WX_i$$

其中：

$$X_i \in \mathbb{R}^{C \times T}, \quad Z_i \in \mathbb{R}^{K \times T}$$

也就是说，CSP 把原来的 22 个 EEG 通道变成若干个空间模式。每个空间模式都是原始电极通道的加权组合。

直观地说，CSP 会寻找这样的通道组合：

$$z(t) = w_1 x_1(t) + w_2 x_2(t) + \cdots + w_C x_C(t)$$

使得某一类任务中该组合信号的方差较大，而另一类任务中方差较小。

16.4 协方差矩阵

对第 i 个 trial，先计算归一化协方差矩阵：

$$\Sigma_i = \frac{X_i X_i^\top}{\text{trace}(X_i X_i^\top)}$$

然后分别对两个类别求平均协方差：

$$\Sigma_0 = \frac{1}{N_0} \sum_{i:y_i=0} \Sigma_i$$

$$\Sigma_1 = \frac{1}{N_1} \sum_{i:y_i=1} \Sigma_i$$

CSP 要找的空间滤波方向 w 满足：投影后在一类中的方差尽量大，在另一类中的方差尽量小。这个目标可以写成：

$$J(w) = \frac{w^\top \Sigma_0 w}{w^\top \Sigma_1 w}$$

最大化或最小化这个比值，就能找到区分两类任务的空间模式。
这个优化问题通常转化为广义特征值问题：

$$\Sigma_0 w = \lambda \Sigma_1 w$$

特征值大的方向对应类别 0 方差较大的空间模式，特征值小的方向对应类别 1 方差较大的空间模式。实际使用时会取两端的若干个空间滤波器组成 W 。

16.5 CSP 输出的特征

CSP 得到投影信号：

$$Z_i = W X_i$$

之后通常取每个空间分量的方差，并取对数：

$$v_{ik} = \log(\text{var}(Z_{ik}))$$

于是每个 EEG trial 不再是一个 $C \times T$ 的时间序列矩阵，而变成一个 K 维特征向量：

$$v_i \in \mathbb{R}^K$$

这就是代码中 `CSP(log=True)` 的含义：用对数方差作为分类特征。

16.6 LDA 的核心思想

LDA 接收 CSP 输出的特征向量 v_i ，学习一个线性分类器：

$$g(v) = a^\top v + b$$

分类规则为：

$$\hat{y} = \begin{cases} 0, & g(v) < 0 \\ 1, & g(v) \geq 0 \end{cases}$$

LDA 的目标是找到一个投影方向，使两类样本的均值分得尽量远，同时类内离散程度尽量小。

若两类 CSP 特征的均值分别为：

$$\mu_0, \quad \mu_1$$

类内散度矩阵为：

$$S_w = \sum_{i:y_i=0} (v_i - \mu_0)(v_i - \mu_0)^\top + \sum_{i:y_i=1} (v_i - \mu_1)(v_i - \mu_1)^\top$$

则 LDA 的典型投影方向可写为：

$$a = S_w^{-1}(\mu_1 - \mu_0)$$

也就是说，LDA 学的是一个能把两类 CSP 特征尽量分开的线性边界。

16.7 从 EEG 到分类结果的完整链路

当前实验中，CSP + LDA 的完整链路可以写成：

$$X_i \in \mathbb{R}^{C \times T} \xrightarrow{\text{CSP}} v_i \in \mathbb{R}^K \xrightarrow{\text{LDA}} \hat{y}_i$$

对应到代码就是：

```
clf = make_pipeline(
    CSP(n_components=6, reg=None, log=True, norm_trace=False),
    LinearDiscriminantAnalysis()
)
```

`make_pipeline` 的意义是把 CSP 和 LDA 串成一个整体。训练时，CSP 先从训练集 EEG 中学习空间滤波器 W ，再把 EEG 转换成特征；LDA 再从这些特征中学习分类边界。

16.8 交叉验证中的训练过程

运行：

```
scores = cross_val_score(clf, X, y, cv=cv, scoring="balanced_accuracy")
```

时，每一折都会执行：

1. 用训练集估计 CSP 空间滤波器 W 。
2. 用训练集的 CSP 特征训练 LDA 分类边界。
3. 把测试集 EEG 输入同一个 CSP，得到测试特征。
4. 用 LDA 对测试特征分类。
5. 计算测试集准确率。

重要的是：CSP 必须只在训练集上学习，不能先对全部数据拟合 CSP 再交叉验证。把 CSP 和 LDA 放进 Pipeline 中，就是为了避免这种数据泄漏。

16.9 在 feet 任务中的解释

如果任务是：

```
left_hand vs feet
```

那么 CSP 学到的是能够区分“左手运动想象”和“脚部运动想象”的空间模式。它可能更关注 C3/C4 与 Cz 附近感觉运动区之间的差异。

如果任务是：

```
right_hand vs feet
```

则 CSP 学到的是“右手运动想象”和“脚部运动想象”的空间差异。

需要强调的是：在 BNCI2014_001 中，feet 只有一个类别。因此 CSP + LDA 只能学习“脚部类 vs 另一个类”的差异，不能学习左脚和右脚之间的差异。

16.10 与 SCI 下肢课题的关系

对于脊髓损伤下肢运动想象解码，CSP + LDA 可以作为第一条传统基线。它的意义不是证明患者真的完成了下肢运动，而是检测 EEG 中是否存在与下肢运动想象相关、可被分类器稳定利用的感觉运动节律差异。

在后续更接近 SCI 的实验中，可以把标签替换成：

```
gait_imagery vs rest  
left_leg vs right_leg  
attempted_movement vs rest
```

但基本数学链路仍然相同：

EEG trial → 空间特征 → 分类器 → 运动意图标签

16.11 如果必须做左右腿

如果研究问题明确要求 左腿 vs 右腿，需要寻找带有左右腿、左右脚、踝关节或下肢单侧运动标签的数据集，或者自己设计采集实验。此时要重点确认：

- 数据标签是否明确区分左腿和右腿。
- 是否是运动想象、实际运动，还是运动执行。
- 是否同步记录 EMG 或运动学信号，用于排除肌电和动作伪迹。
- 电极布局是否覆盖 Cz、FCz、CPz 及周围感觉运动区域。
- 试次数量是否足够进行交叉验证。

在没有左右腿标签的数据集中，不能通过代码把 `feet` 强行拆成左腿和右腿；那样没有真实标签支撑，分类结果没有实验意义。

17 下一步建议

建议下一步新建一个 Notebook，文件名可以为：

```
F:\eeg_bci\projects\motor_imagery_baseline.ipynb
```

先完成 BNCI2014_001 的数据加载、事件查看和一个 CSP + LDA 二分类基线。这样后续无论是写课程项目、论文复现，还是接真实 EEG 设备，都会更清楚每一步在做什么。